

INSTITUTO DE DESARROLLO ECONÓMICO E INNOVACIÓN

Año: 2025



Universidad Nacional de Tierra del Fuego,
Antártida e Islas del Atlántico Sur.

PROGRAMA DE LA ASIGNATURA:

Paradigmas y Lenguajes de Programación
(IF020)

CÓDIGO: IF020

AÑO DE UBICACIÓN EN EL PLAN DE ESTUDIOS:

4 año

FECHA ULTIMA REVISIÓN DE LA ASIGNATURA:

2024-11-30

CARRERA/S: Licenciatura en Sistemas 049/2017,

CARÁCTER: CUATRIMESTRAL (1ro)

TIPO: OBLIGATORIA

NIVEL: GRADO

MODALIDAD DEL DICTADO: PRESENCIAL

MODALIDAD PROMOCION DIRECTA: SI

CARGA HORARIA SEMANAL: 8 HS

CARGA HORARIA TOTAL: 120 HS

EQUIPO DOCENTE

Nombre y Apellido	Cargo	e-mail
Matías Gel	Profesor Adjunto	mgel@untdf.edu.ar
Martin Villarreal	Profesor Adjunto	mvillarreal@untdf.edu.ar

1. FUNDAMENTACION

El estudio de los conceptos de lenguajes y la realización de las actividades propuestas en esta asignatura contribuyen significativamente al desarrollo de capacidades cognitivas superiores en el alumno. Estas incluyen la habilidad para analizar y resolver problemas complejos, razonar de manera crítica y tomar decisiones informadas en el contexto del desarrollo de software, permitiéndole aplicar sus conocimientos de forma práctica y efectiva.

En el marco del Plan de Estudios, la materia aporta a la comprensión profunda de los distintos modelos de computación y paradigmas de programación, así como al conocimiento detallado y análisis de la estructura de los lenguajes de programación, sus construcciones y descripciones. De este modo, el alumno adquiere la capacidad de evaluar y comparar diversos lenguajes, enriqueciendo los conceptos previamente adquiridos en materias relacionadas con el desarrollo de software desde una perspectiva no abordada hasta esta instancia de la carrera.

Considerando la ubicación de la materia en el cuarto año del plan de estudios, se asume que el alumno ya ha incorporado conceptos fundamentales de programación, análisis y diseño, sistemas de información y lenguajes, los cuales constituirán la base sobre la cual se desarrollará el trabajo en esta asignatura.

En articulación con la materia "Taller de Nuevas Tecnologías", que se imparte en paralelo, se introducen los conceptos de programación funcional, los cuales no son cubiertos en otras materias de la carrera. Esto proporciona al alumno las herramientas necesarias para comprender y emplear lenguajes y frameworks multiparadigma utilizados en la mencionada materia. Por otra parte, si bien se cubren los contenidos mínimos establecidos para la asignatura, se otorga una importancia especial al paradigma funcional, dado que muchos lenguajes de programación modernos como Python, Java, Kotlin y JavaScript han incorporado características propias de este paradigma.

2. OBJETIVOS

a) OBJETIVOS GENERALES

Formar en el alumno la capacidad para evaluar de manera crítica y exhaustiva los lenguajes de programación actuales y emergentes desde diversas perspectivas. Proporcionarle los fundamentos teóricos y prácticos necesarios para seleccionar y aplicar los lenguajes más apropiados según las características y requerimientos de los problemas a resolver en el desarrollo de software, permitiéndole tomar decisiones informadas en la elección de las herramientas más adecuadas para cada solución.

b) OBJETIVOS ESPECIFICOS

Mediante el dictado de la materia se pretende que el alumno sea capaz de:

- Adquirir los conceptos fundamentales sobre los cuales se construyen los diferentes lenguajes de programación, comprendiendo su estructura, sintaxis y semántica.
- Evaluar y comparar lenguajes de programación en términos de desempeño, eficiencia y adecuación para determinados propósitos y áreas de aplicación.
- Desarrollar criterios sólidos para realizar evaluaciones críticas de lenguajes de programación, basados en fundamentos teóricos y prácticos.
- Seleccionar el lenguaje de programación más apropiado para el desarrollo de aplicaciones específicas, considerando los requerimientos del proyecto y las características del lenguaje.
- Incrementar la capacidad para abordar problemas reales con enfoques diversos, fomentando el pensamiento crítico y la flexibilidad en la resolución de problemas.
- Utilizar de manera eficiente los lenguajes de programación, aprovechando sus características y herramientas para optimizar el desarrollo de software.
- Comprender el rol esencial de los lenguajes de programación en el proceso de desarrollo de software, reconociendo su impacto en la arquitectura, mantenimiento y evolución de los sistemas.
- Analizar comparativamente distintos paradigmas de programación, con un énfasis especial en la programación funcional, entendiendo sus ventajas, limitaciones y ámbitos de aplicación.
- Adquirir conocimientos sobre los mecanismos y herramientas que los lenguajes proporcionan para implementar las características de cada paradigma, aplicándolos en situaciones prácticas.

3. CONDICIONES DE REGULARIDAD Y APROBACION DE LA ASIGNATURA

Entrega de Trabajos Prácticos:

? Los estudiantes deberán entregar oportunamente todos los trabajos prácticos asignados durante el curso, cumpliendo con los requisitos establecidos por la cátedra.

Evaluación de Parciales

Primer Parcial:

? Consistirá en un trabajo de programación relacionado con los paradigmas estudiados. Este trabajo será de carácter práctico, tendrá una duración de una semana y deberá ser defendido oralmente por el alumno. Esta evaluación asegura que los estudiantes no solo apliquen lo aprendido, sino que también puedan justificar sus decisiones y argumentar sus enfoques fomentando el razonamiento crítico.

Segundo Parcial:

? Se basará en la preparación y presentación de un lenguaje de programación elegido por el alumno de entre los propuestos por la cátedra. El alumno deberá exponer el lenguaje aplicando los conceptos adquiridos en la materia. Y entregar el material utilizado en la presentación. Este parcial permite evaluar la capacidad de análisis de lenguajes, promoviendo la capacidad de investigación y el dominio del contenido, habilidades esenciales en la formación profesional.

Aprobación de Parciales:

? Para aprobar cada parcial, el alumno deberá obtener una calificación no menor a cuatro (4) sobre un total de diez (10). En caso de no alcanzar esta nota, tendrá una instancia recuperatoria.

Calificación Final de la Cursada

? La nota final será el promedio de las calificaciones obtenidas en los exámenes parciales.

? Para aprobar la cursada, la nota final no podrá ser inferior a seis (6) sobre diez (10).

Promoción de la Asignatura

Además de cumplir con las condiciones básicas de cursada, para promocionar la asignatura se requiere:

? Aprobación de los dos parciales prácticos en la primera instancia, con una calificación igual o superior a siete (7) sobre diez (10).

? Si el alumno se encuentra en condición de promoción, deberá rendir un examen oral en el cual responderá preguntas sobre el contenido teórico de la materia y en relación con el lenguaje presentado como segundo parcial. Este examen deberá ser aprobado con una calificación igual o superior a siete (7) sobre diez (10). El examen oral permite validar el conocimiento adquirido y la capacidad de argumentación técnica. Promoviendo la comunicación oral. Se realiza en el contexto del lenguaje seleccionado en el segundo parcial y con preguntas orientadas a la aplicación del lenguaje en dominios prácticos y fundamentación de tales decisiones.

Aprobación en Condición Libre

Para aprobar la materia en condición de alumno libre, el estudiante deberá rendir:

? Una instancia práctica, resolviendo ejercicios en los tres paradigmas estudiados, según el programa.

? Un examen teórico que abarque todos los temas del programa de la asignatura.

Este esquema asegura que los estudiantes en condición libre cumplan con los mismos estándares académicos que los regulares, garantizando una evaluación integral de sus conocimientos y habilidades.

4. CONTENIDOS DE LA ASIGNATURA

Contenidos mínimos

- Sintaxis y semántica. Nociones básicas de semántica formal. Semántica operacional.
- Lenguajes de programación: entidades y ligaduras.
- Sistemas de tipos.
- Niveles de polimorfismo.
- Encapsulamiento y abstracción.
- Conceptos de intérpretes y compiladores.
- Criterios de diseño y de implementación de lenguajes de programación.
- Paradigmas de programación: imperativo, orientado a objetos, funcional, lógico.
- Concurrencia y paralelismo.

Programa Analítico

MÓDULO I: Conceptos introductorios

Razones e importancia del estudio de lenguajes de programación. Síntesis de los paradigmas de programación. Paradigma imperativo. Conceptos fundamentales. Paradigma Declarativo.

Conceptos fundamentales. Aportes de cada paradigma. Dominios de aplicación. Lenguajes característicos de los diferentes dominios. Criterios para el estudio, análisis, selección y evaluación de lenguajes. Evolución de los lenguajes de programación. Perspectiva histórica. Introducción a Kotlin como lenguaje multiparadigma.

MÓDULO II: Paradigma de Programación funcional.

Introducción a la programación funcional. Fundamentos de la programación funcional. Importancia e impacto en los lenguajes y frameworks actuales.

Funciones Puras. Inmutabilidad. Recursividad. Ventajas y desventajas del paradigma funcional. Dominios de aplicación. Kotlin como lenguaje funcional. Comparación con lenguajes funcionales puros.

Expresiones lambda y funciones anónimas, implementación en Kotlin y comparación con otros lenguajes. Funciones de orden superior utilizando Kotlin. Clausuras y su ámbito léxico.

Composición de funciones. Aplicación parcial de funciones y currying. Evaluación perezosa con aplicaciones en Kotlin. Manejo de efectos secundarios. Breve introducción a mónadas con Kotlin.

Tendencias actuales de la programación funcional con ejemplos en diferentes lenguajes con mecanismos de soporte a la programación funcional. Programación funcionales reactiva.

MÓDULO III: Paradigma de programación lógica utilizando Prolog como lenguaje representativo. Fundamentos de la programación lógica. Sintaxis y las estructuras básicas de Prolog, Hechos, reglas y consultas. Variables, unificación y backtracking, Modelado de bases de conocimiento. Consultas complejas. Dominios de aplicación.

MÓDULO IV: Paradigma de Programación orientada a objetos

Fundamentos de la programación Orientada Objetos. Estudio y práctica en Lenguaje Kotlin como representativo del paradigma OO. Ejemplos comparativos en diferentes lenguajes OO como Python, Java, Typescript.

MÓDULO V: Sintaxis y semántica de Lenguajes de Programación

Sintaxis. Criterios generales. Elementos sintácticos de un Lenguaje. Descripción de la sintaxis de un LP. Definición formal de la sintaxis de un lenguaje. Gramática formal: Gramática BNF. BNF extendido. Árboles sintácticos. Diagramas de Sintaxis. Sintaxis abstracta y concreta. Ambigüedad. Semántica. Semántica estática. Gramática de atributos. Herramientas para la descripción semántica de un lenguaje.

Procesadores de Lenguajes. Métodos de interpretación y compilación. Etapas de un proceso de traducción.

MÓDULO VI: Variables

Variables. Atributos: Nombres y ámbito de nombres. Valor izquierdo. Valor derecho. Tiempo de vida. Tipos. Ámbito. Concepto de Binding. Tiempo de vinculación. Binding de tipos: Binding estático y dinámico de tipos. Binding de almacenamiento y tiempo de vida. Variables estáticas. Variables dinámicas de pila. Variables dinámicas de heap. Chequeo de tipos. Tipado fuerte. Análisis de lenguajes fuertemente tipados. Compatibilidad de tipos. Compatibilidad nominal y estructural. Subtipo. Tipo derivado. Ámbito. Ámbito estático y dinámico. Evaluación de ámbito estático y dinámico. Entorno de referencia. Inicialización de variables. Variables no inicializadas. Asignación estática y dinámica de memoria. Ejemplos en kotlin y comparación con otros lenguajes.

MÓDULO VII: Tipos de Datos

Tipos de datos. Tipos built-in y primitivos. Tipos ordinales definidos por el usuario. Tipos agregación y su implementación: Producto cartesiano. Uniones y uniones discriminadas. Mapeos Finitos. Arrays. Categorías de arrays: Estáticos, de pila dinámica, dinámicos de heap. Tipos Secuencia. Decisiones de diseño en Strings. Conjunto Potencia. Tipos Recursivos. Tipo Puntero. Inseguridad de los punteros. Punteros colgantes. Recolección de basura.

Sistemas de tipos. Tipos monomórficos vs. Tipos polimórficos. Tipos de datos que aceptan null y sus operadores. Clases. Lenguajes fuertemente tipados
Ejemplos en kotlin y otros lenguajes .

MÓDULO VIII: Estructuración de cómputos: Expresiones y estructuras de control
 Expresiones. Expresiones aritméticas, relacionales y booleanas. Reglas de Precedencia. Asociatividad. Paréntesis. Sentencias de asignación. Asignación como expresión. Asignación en modo mixto. Evaluación corto-circuito vs, Evaluación estricta. Sobrecarga de operadores. Conversiones de tipo. Coerciones. Conversión de tipo explícita. Estructuras de control. Enunciados compuestos. Enunciados de selección. Selectores anidados. Selección Múltiple. Enunciados iterativos. Ejecución condicional. Mecanismos de control de bucle. Iteradores y generadores. Ejemplos de expresiones y estructuras de control en kotlin y otros lenguajes .

MÓDULO IX: Manejo de Excepciones
 Estructuras de control de Nivel Unidades. Gestión de excepciones. Eventos excepcionales. Programa tolerante a fallos. Lanzamiento de una excepción. Manejador de excepciones. Propagación de una excepción. Formas de manejar excepciones. Facilidades de lenguajes para el manejo de excepciones. Manejo de excepciones en Kotlin y otros lenguajes. Comparaciones.

MÓDULO X: Estructuración de programas
 El concepto de abstracción procedural. Fundamentos de Subprogramas. Parámetros. Procedimientos y Funciones. Métodos de paso de parámetros. Modos de implementación. Cuestiones de Diseño. Métodos utilizados por los distintos lenguajes. Subprogramas como parámetros. Tipos de valores de retorno. Subprogramas sobrecargados. Subprogramas polimórficos. Implementación de subprogramas. Soporte de modularidad. Encapsulación. Interface e Implementación. Separación de Interface e Implementación. Compilación separada e independiente. Librerías de módulos. Características de diferentes lenguajes para la organización de programas: Cómo se provee encapsulación, interface e implementación, organización general del programa. Facilidades de compilación. Estructuras de datos genéricas. Ejemplos en kotlin y otros lenguajes.

MÓDULO XI: Concurrencia
 Concurrencia. Conceptos fundamentales de concurrencia a nivel de subprogramas. Niveles de concurrencia. Threads. Sincronización de cooperación y competencia. Comunicación entre procesos. Procesos. Tareas. Programación asíncrona, Ejemplos en kotlin y otros lenguajes.

5. RECURSOS NECESARIOS

- Proyector
- Pc
- Se recomienda La Compra De La Versión Actualizada Del Libro "Concepts Of Programming Languages, 12ª Ed. ". Y La Compra De Los Libros "Programming Languages: Principles And Paradigms, Second Edition" , "Programming Languages Principles And Practice, 3rd Ed.". Los Cuales Se Encuentran En La Bibliografía Del Programa.

6. PROGRAMACIÓN SEMANAL

Semana	Unidad / Módulo	Descripción	Bibliografía
1	I	Conceptos introductorios de Lenguajes. Práctica I: Preguntas de conceptos introductorios, practica de introducción al lenguaje Kotlin.	Kenneth, Gabbrielli, Sebesta
2	II	Introducción a la programación funcional con Kotlin. Practica II: Programación funcional en Kotlin	Kenneth, Gabbrielli, Sebesta, Vermeulen

3	II	Aspectos avanzados de Programación funcional..Practica II: continuan practica con Kotlin	Kenneth, Gabbrielli, Sebesta, Vermeulen
4	II	Introducción a mónadas ejemplos en Kotlin. Practica II: continua por practica con Kotlin.	Vermeulen
5	III	Aplicación de programación funcional en otros lenguajes. Practica III: Resolución de problemas simples aplicando programación funcional en Python y Javascript.	Kenneth, Kent, Sebesta
6	IV	Paradigma de Programación Lógica. Conceptos básicos e Introducción a PROLOG. Práctica: Consultas de Bases de datos en PROLOG	Kenneth, Kent, Sebesta, Spivey
7	IV	Introducción Paradigma OO. Práctica: Clases y Objetos y poliformismo en Kotlin.	Kenneth, Bramer
8	V	Presentación Trabajo final sobre Paradigmas de LP. Parcial Práctico N° 1	Kenneth, Bramer
9	III-V	Sintaxis y semántica de Lenguajes de Programación. Practica IV: Practica de mecanismos para descripción de la sintaxis y semántica.	Kenneth, Gabbrielli, Sebesta
10	VI	Conceptos de Variables, binding, ámbito. Práctica en Laboratorio. Ejercicios de binding y ámbito en kotlin y en diferentes lenguajes.	Kenneth, Gabbrielli, Sebesta
11	VII	Tipos de Datos y estructuración. Práctica en Laboratorio. Ejercicios con estructuras de datos en diferentes lenguajes.	Kenneth, Guezzi, Pratt; Sethi
12	VIII	Estructuras de control. Práctica en Laboratorio: Expresiones y estructuras de control en diferentes lenguajes.	Kenneth, Guezzi, Pratt; Sethi
13	IX	Manejo de excepciones. Práctica en Laboratorio: Ejercitación sobre manejo de excepciones comparando diferentes lenguajes.	Kenneth, Guezzi, Pratt; Sethi
14	X	Abstracción Procedural, Modularidad, Genericidad Práctica en Laboratorio: Implementación de subprogramas, módulos y unidades genérica.	Kenneth, Guezzi, Pratt; Sethi
15	XI	Concurrencia paralelismo. Práctica en Laboratorio: Concurrencia en diferentes lenguajes.	Kenneth, Guezzi, Pratt; Sethi
16	VI-XI	Presentación oral de Trabajos sobre lenguajes de programación Evaluación y comparación de lenguajes. Parcial Práctico N° 2	
17	I-Xi	Trabajo Final para promoción - Reunión de cátedra para evaluación del dictado de la cursada y ajuste de contenidos. Elaboración del Informe de cátedra	

7. BIBLIOGRAFIA DE LA ASIGNATURA

Autor	Año	Título	Capítulo/s	Lugar de la Edición	Editor / Sitio Web
Louden, Kenneth C	2011	Programming Languages Principles and Practice, 3rd ed.	1-13	EEUU	Cengage Learning;
Maurizio Gabbrielli , Simone Martini	2023	Programming Languages: Principles and Paradigms, Second Edition	1-12	Suiza	Springer Nature Switzerland AG
Robert Sebesta	2019	Concepts of Programming Languages, 12ª ed.	1-16	USA	Pearson
Michael Spivey	2005	An introduction to logic programming through Prolog	1-7 complementaria	Reino Unido	https://spivey.orient.ox.ac.uk/corner/An_introduction_to_logic_programming_through_Prolog
Marco Vermeulen, Rúnar Bjarnason, Paul Chiusano	2021	Functional Programming in Kotlin	1-13	USA	Manning Publications Co.
Documentación Oficial Kotlin	2024	Documentación Oficial Kotlin		Documentación Oficial Kotlin	https://kotlinlang.org/docs/learning-materials-overview.html

Firma del docente-investigador responsable

VISADO		
COORDINADOR DE LA CARRERA	DIRECTOR DEL INSTITUTO	SECRETARIO ACADEMICO UNTDF
Fecha :	Fecha :	

Este programa de estudio tiene una validez de hasta tres años o hasta que otro programa lo reemplace en ese periodo